

FOCUSBUDDY

PRODUCTION AUDIT REPORT V3

Post-Infrastructure Re-Audit
Date: 2026-03-15

V1	V2		V3
24	38	→	62

SIGNIFICANT PROGRESS — NOT YET PRODUCTION READY

EXECUTIVE SUMMARY

Fix	Before	After
Test Coverage	0 tests, 0% coverage	190 tests, 14 suites, 81% service coverage
ESLint	Not installed, stub script	ESLint 9 configured, 0 errors, 49 warnings
CI/CD	Facade (stub lint, empty test)	Real 5-job pipeline: typecheck+lint+test+security+build
Error Monitoring	None	Sentry with user context, breadcrumbs, header scrubbing
Rate Limiting	TOCTOU race condition	Atomic PG function with FOR UPDATE row locking
CORS	Wildcard * on all endpoints	Environment-based ALLOWED_ORIGIN on all functions

Score improved from 38 to 62. The app has moved from 'proof-of-concept' to 'early-stage product with known gaps.'

TEST SUITE — 190 TESTS, 14 SUITES

Suite	Tests	Statements	Lines
utils/time.ts	32	100%	100%
utils/constants.ts	18	100%	100%
services/auth	14	96%	100%
services/shields	13	100%	100%
services/checkin	8	100%	100%
services/goals	8	91%	100%
services/sessions	13	93%	100%
services/promises	13	93%	100%
services/profile	12	95%	100%
services/chat	13	96%	100%
stores/authStore	16	69%	73%
stores/sessionStore	14	76%	79%
hooks/useAppState	7	100%	100%
lib/sentry	9	mock-based	validation

Aggregate service coverage: 81.5% statements, 85.6% lines. All critical paths tested: auth, shields (CAS), check-in, sessions, promises/trust score, chat rate limiting.

INFRASTRUCTURE IMPROVEMENTS

ESLint Configuration

- ESLint 9 with flat config (eslint.config.mjs)
- Plugins: @typescript-eslint, react, react-hooks, react-native
- Results: **0 errors, 49 warnings** (all no-explicit-any or no-unused-vars)
- CI enforces max 50 warnings — above this, builds fail

CI/CD Pipeline (5 Jobs)

Job	Command	Purpose
1. typecheck	tsc --noEmit	Catches type errors
2. lint	eslint . --max-warnings 50	Enforces code quality
3. test	jest --coverage --ci	Runs 190 tests + 'no tests found' guard
4. security	npm audit + secret scan	Catches vulnerabilities + leaked keys
5. build-check	expo export --platform web	Verifies app builds (depends on 1-3)

Sentry Error Monitoring

- @sentry/react-native installed and configured
- Wrapper: initSentry(), captureError(), setSentryUser(), addBreadcrumb()
- Initialized at module level in _layout.tsx (before any component renders)
- User context set on auth state changes for error attribution
- Authorization/Cookie headers scrubbed from error events
- Graceful no-op when DSN not set (safe for development)

SECURITY FIXES

Atomic Rate Limiting

- New PostgreSQL function **increment_rate_limit()** in 002_rate_limit_function.sql
- Uses **SELECT ... FOR UPDATE** row locking — true atomic check-and-increment
- Handles window expiration, row creation, and limit enforcement in single transaction
- Returns -1 when rate limited, positive count on success
- ai-chat/index.ts calls .rpc('increment_rate_limit') instead of old CAS pattern
- **No more TOCTOU race condition** — impossible to bypass under concurrent load

CORS Restriction

- All 4 edge functions use **Deno.env.get('ALLOWED_ORIGIN')** with fallback
- No more Access-Control-Allow-Origin: '*' anywhere in codebase
- revenuecat-webhook: no CORS headers on non-OPTIONS responses (server-to-server)
- Verified: grep confirms only allowedOrigin references remain

REMAINING ISSUES

CRITICAL (3 remaining)

ID	File	Issue	Effort
C1	trial-confirmation.tsx	Client-side subscription INSERT	0.5 day
C2	paywall.tsx	Purchase flow stubbed ('Coming Soon')	2-3 days
C3	subscriptionStore.ts	RevenueCat never calls configure()	1 day

HIGH (7 remaining)

ID	File	Issue
H1	ai-chat, ai-checkin	Prompt injection — user data unescaped in system prompts
H2	ai-chat, ai-checkin	Timezone bug — UTC dates instead of user timezone
H3	ai-checkin	Silent task insert failure — 200 OK but tasks not saved
H4	revenuecat-webhook	JSON.parse no try/catch — crashes on bad payload
H5	revenuecat-webhook	appId may not be valid UUID
H6	check-in.tsx:160	UUID uses Math.random() — collisions possible
H7	database.ts	deleted_at field missing from types

MEDIUM (10) + LOW (6) — see full markdown report for details

PRODUCTION READINESS SCORE

62/100

SIGNIFICANT PROGRESS — APPROACHING PRODUCTION

Category	Weight	V1	V2	V3	Weighted
Security	25%	20	35	60	15.00
Correctness	20%	30	55	60	12.00
Test Coverage	15%	0	0	65	9.75
Infrastructure	15%	5	15	70	10.50
Performance	10%	45	45	50	5.00
Code Quality	10%	50	60	72	7.20
UX Completeness	5%	35	55	55	2.75
TOTAL	100%	24	38	62	62.20

ROADMAP TO 90+

#	Requirement	Effort	Impact
1	Implement RevenueCat purchase flow	2-3 days	+10
2	Move subscription creation server-side	0.5 day	+4
3	Add component tests (10-15 files)	2 days	+5
4	Escape user data in AI prompts	0.5 day	+3
5	Fix timezone handling	0.5 day	+2
6	Add React error boundaries	0.5 day	+2
7	Fix remaining HIGH bugs	1 day	+4
8	Add Prettier + husky pre-commit	0.5 day	+1
9	Fix EAS projectId + build config	1 hour	+1

Total: ~7-9 days to reach 90+. Primary blocker: payment stack (RevenueCat).

FINAL ASSESSMENT

The six infrastructure fixes transformed FocusBuddy from a proof-of-concept to an early-stage product. The app now has real safety nets: **190 tests catch regressions**, **ESLint enforces quality**, **CI blocks broken merges**, **Sentry catches production errors**, **rate limiting is truly atomic**, and **CORS is restrictive**.

The remaining gap to 90+ is primarily the **payment stack** (RevenueCat initialization, purchase flow, server-side subscription management) and secondary hardening (prompt escaping, timezone fixes, error boundaries).

The architecture, code quality, and infrastructure are now approaching production standards. With 7-9 more days of focused engineering, this app can reach 90+.

Score trajectory: 24 → 38 → 62. Projected: 90+ with ~7-9 days work.

End of Audit Report V3